

Reason

A Proof Language of Mathematical Intent

Concise mathematical narratives, explicit reconstruction contracts, and kernel-checked evidence

A proposal grounded in *ProveIt* and *Leant*

5 September 2026

Abstract. Formal proofs often mix their mathematical argument with representation changes, theorem-discovery choices, and instructions to a particular elaborator. Merely shortening tactic scripts does not reliably separate these concerns. This article proposes **Reason**, a declarative layer over Lean in which authors write a typed mathematical argument and delegate explicitly delimited reconstruction tasks. Its principal unit is a *claim with a contract*: a fixed proposition, an admissible context, a mathematical method or library view, and a resource policy. Untrusted automation may search widely, but only checked proof terms establish claims. The design combines scoped proof plans, certified changes of mathematical representation, local synthesis, semantic ambiguity controls, and search-free publication artifacts.

The proposal is developed through examples from the ProveIt repository: reflection symmetry, factorial decomposition, affine finite differences, local derivative identities, and sums of integer square roots. Leant contributes concrete precedents for candidate synthesis, contextual tactic suggestions, bounded search, and preservation of candidate identity; its existing native-Lean rewrite study is an important architectural precursor. We give mathematical proofs for the worked examples, a small core semantics with a relative soundness argument, implementation and evaluation plans, and an executable toy checker illustrating the evidence boundary. No implementation of the full language, Lean compilation of the proposed syntax, or measured reduction in formalization effort is claimed.

Status. A research and engineering proposal, not a new trusted logic. **Reason** is a provisional label. Repository observations concern selected files on `main` accessed on 5 September 2026, not a complete repository audit or a reproducibly pinned build.

Contents

1	The objective: shorter explanations, not weaker theorems	2
2	What the repositories actually suggest	3
3	The language concept	4
4	Mathematical views and disciplined omission	6
5	Worked example I: reflection symmetry through a normal form	7
6	Worked example II: a factorial identity and honest abstraction cost	8
7	Worked example III: finite differences and certified reindexing	9
8	Worked example IV: differentiating a local orbit identity	11
9	Worked example V: integer square roots by double counting	12
10	A small core semantics	14
11	Ambiguity, intent, and the limits of inference	16
12	Synthesis architecture inspired by Leant	17
13	Trust, publication, and reproducibility	19
14	The reader's interface: proof lenses and mathematical diagnostics	21
15	Relationship to existing proof languages and tools	22
16	Implementation strategy and migration	23
17	Evaluation: what would count as success?	24
18	Executable companion: a deliberately small evidence model	25
19	Risks, open design questions, and conclusions	26
A	A compact grammar and an elaboration trace	27
B	Failure cases and required responses	28
C	Source map and reproducibility notes	29
	References	30

1 The objective: shorter explanations, not weaker theorems

A mathematician may write, “Split the subsets according to whether they contain the last index; the two sums give the induction step.” A formal implementation may additionally introduce a disjointness proof, an injectivity proof for insertion, membership conversions, two sum-rewriting lemmas, an equality in the exponent, and a normalization invocation. The mathematical sentence is not wrong. It describes a reusable argument pattern whose detailed realization depends on the representation of finite sets and sums.

The desirable abstraction is therefore not a permission to omit arbitrary proofs. It is a way to identify the intended pattern, supply its mathematical parameters, and require a machine to reconstruct the corresponding evidence. Some apparently routine details are genuinely necessary: deleting a denominator requires a nonzero hypothesis, differentiating a local identity requires neighborhood equality, and replacing natural subtraction by rational subtraction requires an order bound. A successful language suppresses repeated *presentation* of such details without suppressing their *verification*.

Reason is proposed as a layer over Lean, rather than a replacement for its dependent type theory. The theorem statement remains a precisely elaborated Lean proposition. Every accepted proof ultimately supplies a term of that proposition in a fixed environment. This reuses the distinction already present in Lean between flexible proof construction and a small checking boundary [13, 14].

1.1 Four different quantities that should not be confused

Source length is the number of characters or lines the author writes. *Explanatory length* measures how much of the mathematical argument the reader must understand. *reconstruction cost* measures the work needed to find the omitted evidence. *checking cost* measures the work needed to validate that evidence. These quantities can move in opposite directions.

A one-line invocation of an enormous search procedure can minimize source length while hiding the proof and making maintenance unpredictable. Conversely, a six-line argument that names an intermediate invariant may be more useful than a two-line proof even when both elaborate immediately. The design objective is not the smallest script. It is a readable, stable proof plan whose omitted steps can be reconstructed at acceptable cost and replayed without rediscovery.

A useful optimization model is

$$J = \alpha L_{\text{author}} + \beta C_{\text{reconstruct}} + \gamma C_{\text{check}} + \delta C_{\text{repair}} + \eta C_{\text{reader}},$$

subject to fixed theorem meaning and an explicit trust policy. The weights are project choices, not universal constants. A teaching environment, a research notebook, and a long-lived library will reasonably choose different weights.

1.2 What this article contributes

The proposal centers on an integration of five ideas: a mathematical proof skeleton with ordinary lexical scope; a reconstruction contract attached to each nontrivial omission; certified library views for recurring representation changes; a strict distinction between a fixed statement and a variable proof search; and durable evidence that can be checked without a model or search service. None of declarative proofs, tactic automation, or proof-term synthesis is new in isolation. The specific design question is how to make their interfaces cooperate without losing either mathematical readability or auditability.

The worked examples are not hypothetical complaints about formal mathematics. They isolate concrete operations appearing in the selected ProveIt files, then develop alternative presentation and reconstruction strategies. The alternatives are designs, not experimentally validated speedups. Where a library lemma would be required, its cost is made visible.

2 What the repositories actually suggest

2.1 Scope and limits of the inspection

The reviewed ProveIt material comprises the basic definitions, arithmetic lemmas, and affine-difference orbit module in the Fabius development, together with the integer-square-root summation module in the integer-sums development. Appendix C gives their exact paths. The inspected toolchain file names Lean 4.32.0 [5]. These are deliberately varied examples, not a random sample. No whole-repository build, tactic profiler, declaration-count analysis, or search for all alternative proofs was performed.

The Leant inspection includes its README, synthesis-internals and candidate-quality documents, two Haskell source modules, and the existing Lean-native rewrite analysis. The distinction between these sources matters: a design document describes a proposed architecture, whereas a source module shows a concrete implementation boundary. A successful callback in that implementation is not automatically an independently inspected Lean kernel receipt.

The repository branches were inspected through public web views. An immutable commit identifier was not established in this session. The source map in Appendix C therefore records exact paths and theorem names, with the access date, but does not pretend to provide a pinned experimental snapshot.

2.2 ProveIt: useful mathematics already separated from some plumbing

The basic Fabius file distinguishes a bounded function, its real-valued coercion, and a folded Rvachev-style function. Several elementary range and support results are already short. The evenness proof for `rvachevUp`, however, explicitly handles signs, unfolds branches, and normalizes arguments. This suggests a reusable “fold through absolute value” view; it does not suggest that every theorem needs stronger search [1].

The arithmetic file already extracts small recurring lemmas, including a binomial-coefficient recurrence and a factorial/odd-product identity. Its factorial proof uses induction and targeted rewrites before ring normalization. This is a positive baseline: concise mathematical language should build on well-factored libraries, not compare itself only with deliberately unoptimized Lean [2].

The affine-difference file exposes two different sources of verbosity. A powerset formula requires bookkeeping for insertion, disjointness, and finite sums. A derivative formula requires preservation of a local domain and eventual equality near a point. The former admits a reusable finite-index partition interface; the latter includes hypotheses whose visibility is mathematically important [3].

The integer-square-root file distinguishes a rational algebraic formula from the public natural-number formula. It proves the divisibility and order facts needed to relate them. A language that automatically “works over the rationals” must preserve precisely these obligations [4].

2.3 Leant: synthesis is useful only with a disciplined boundary

Leant’s documented workflow includes structural term synthesis and advisory tactic suggestions checked against a contextual proof state. Its synthesis pipeline translates a supported structural fragment, considers candidate terms, and re-elaborates candidates against the actual query. The tactic-suggestion workflow need not mutate the accepted script merely because a suggestion was found [7].

Three implementation lessons are especially relevant. Candidate identity must include its originating query and context, not merely the text eventually printed. Structural simplification and ranking must precede verification without becoming authorities for correctness. A bounded miss, an unsupported translation, and a checked candidate are different outcomes. The synthesis-internals and candidate-quality documents explicitly discuss these kinds of boundaries [8, 9].

The source module `Verification.hs` makes a narrower distinction that should survive any redesign:

its private `Verified` wrapper records acceptance by a callback, and its comment explicitly disclaims treating that wrapper as a kernel, solver, or behavioral proof. The type helps enforce a software protocol; the callback and downstream checker still matter [10].

In `Fragment.hs`, the generated verification program checks a candidate as a noncomputable example at the queried type. The same module contains scope-sensitive translation machinery and conservative handling of unsupported dependencies. The practical message is to keep search abstractions subordinate to checking at the original target, not to confuse a convenient search type with the full dependent proposition [11].

Leant’s existing native-Lean rewrite study already proposes a Lean controller/worker architecture, neutral search representations, and direct checking of reconstructed Lean expressions. This article does *not* claim that idea as a new discovery. It develops the additional language layer: the mathematical unit of reconstruction, its scope and explanation contract, and its durable publication form [12].

2.4 A diagnosis with three categories

The inspection motivates a separation between *representational overhead*, *logical structure*, and *mathematical side conditions*. Membership conversions and coercion normalization often belong to the first category. Choosing an induction invariant or a witness belongs to the second. Nonvanishing, summability, local differentiability, and injectivity of an index map belong to the third.

Automation should usually hide the first category after generating evidence. It should usually expose the second, because that is where the author’s argument lives. For the third, the appropriate policy is conditional: discharge obvious obligations quietly, but retain a visible account of the hypothesis and show it whenever it is missing, delicate, or changes the theorem. “Routine” is a user-interface classification, not a logical status.

3 The language concept

3.1 Typed statements with a controlled mathematical narrative

A `Reason` document combines exact mathematical expressions with a small collection of proof commands. Expressions elaborate to Lean terms. Narrative commands introduce binders, state claims, select a proof schema, or request reconstruction. Unrestricted prose can accompany a proof as commentary, but cannot itself change a theorem’s meaning.

Here is a schematic example; all `Reason` listings in this article are *proposed syntax*, not currently executable Lean:

```
theorem application (A B C : Prop):
  (A -> B -> C) -> (A -> B) -> A -> C
proof
  assume f, g, x
  conclude C from f, g, x by structural
end
```

The final step requests a term of the already fixed proposition `C`. Structural synthesis can discover `f x (g x)`. The corresponding ordinary Lean term is

```
fun f g x => f x (g x)
```

The language did not infer a different theorem. It supplied a small missing composition. This example illustrates the useful division of labor between a human choosing the argument’s interfaces and a synthesizer filling an inhabitation problem.

3.2 Claims, not tactics, are the primary units

In a tactic-centric presentation, the visible command often says what the prover should do: rewrite, simplify, apply, or try a tactic. In **Reason**, the primary visible command normally says what mathematical fact is being established:

```
claim hbound: j^2 <= n
  from hj, hs by ordered_arithmetic
```

The reconstruction annotation still specifies a method, but it is subordinate to the claim. A later library change may alter the proof of `hbound` without altering the narrative or the target proposition. The proof editor can expand the annotation into the generated Lean proof when diagnosis is necessary.

Not every step needs an annotation. A project profile may make certain small methods available by default. The crucial constraint is that defaults affect how evidence is found, not which proposition is being proved. A failure should produce an unproved named claim with a useful obligation, not silently change a binder, coerce a different expression, or add a premise to the exported theorem.

3.3 A small core and extensible mathematical methods

The logical core needs relatively few commands: `assume`, `let`, `claim`, `suffices`, `obtain`, `cases`, `induct`, `calc`, and `conclude`. A method such as “partition subsets by their last element” is not a new logical rule. It is a registered schema with typed parameters and a Lean theorem or proof-producing implementation behind it.

This distinction prevents the parser from accumulating hundreds of domain-specific special cases. The core manages contexts and obligations. Libraries contribute mathematical views and proof methods. A finite-sums library can provide a parity partition; an analysis library can provide differentiation under specified hypotheses; a number-theory library can provide exact division transport. Each contribution must document what it requires and what evidence it returns.

3.4 Reconstruction contracts

A reconstruction contract is a tuple

$$\mathcal{C} = (\mathcal{E}, \Gamma, G, H, V, M, B, P),$$

where $\mathcal{E}$ is the fixed declaration environment, Γ the ordered local context, G the exact target, H an admissible premise set, V certified representation views, M permitted methods, B resource limits, and P the trust and explanation policy. The environment includes the actual definitions and instances used for elaboration, not just a list of import names.

For example:

```
claim h: cast_nat (u - v) = cast_nat u - cast_nat v
  from hle
  using view nat_to_rat
  by normalize
  with budget local
```

The view supplies the relevant transport theorem and requires $v \leq u$. The annotation does not authorize arbitrary reinterpretation of subtraction. The contract records the resulting typed expression, the proof of the bound, and the permitted method. Whether the author sees a separate bound line depends on the proof’s intended explanatory granularity.

A contract is *not* an extra axiom or a declaration that a method must succeed. If the method cannot construct the proof, the claim remains open. Its benefit is that the gap is explicit, reproducible, and narrower than “prove whatever this paragraph appears to mean.”

4 Mathematical views and disciplined omission

4.1 A view is a proved interface, not a conversion guess

A mathematical view is a package connecting two representations with certified operations. It may expose an equivalence, an embedding, an equality, a transport theorem, or a collection of conditional rewrite rules. Examples include representing a sign-dependent definition by an absolute-value expression, representing a finite sum through a bijective index map, or embedding natural-number algebra into a field.

A useful view package contains the source and target types, the forward interpretation, the law needed by the current operation, and a list of side conditions. A full inverse is not always needed. An injective embedding is enough to transfer equality back from rationals to naturals, whereas changing variables in a finite sum requires an appropriate bijection or an explicitly multiplicity-aware theorem.

The word “certified” applies to each law actually used. It does not mean that every conceivable transformation involving the two representations is permitted. An embedding of semirings preserves addition and multiplication; that statement alone says nothing about truncated subtraction or integer division.

4.2 Examples of safe and unsafe omission

For $m, n \in \mathbb{N}$, natural subtraction is truncated. Consequently,

$$\iota(m - n) = \iota(m) - \iota(n)$$

is valid in $\mathbb{Q}$ under $n \leq m$, but false without it: take $m = 0, n = 1$. Similarly, the cast of natural division by d agrees with field division when suitable exact-divisibility and denominator conditions hold; a blanket transport is false already at $1/2$.

For a real square root, $\sqrt{x^2} = |x|$, not necessarily x . For a function identity, $f(z) = g(z)$ at one point does not imply $f'(z) = g'(z)$. For finite sums, mapping an index set into another is insufficient unless fibers, multiplicities, or bijectivity are accounted for. These are mathematical distinctions a concise language must preserve, even when their proofs can be reconstructed automatically.

4.3 User-facing behavior

The normal display might show

```
| work in Rat using exact_nat_arithmetic
```

with a small expandable entry indicating that two transport obligations were discharged. If a bound is missing, the display should identify the failed mathematical precondition:

```
| Cannot transport natural subtraction at this expression.
| Required: v <= u.
| Available facts do not establish this obligation.
| No change has been made to the theorem statement.
```

It should not merely report a failed cast tactic, nor assume the bound because the intended formula looks plausible.

4.4 Where a view should live

A view should normally be a reusable library object rather than a hidden model-generated patch. A local one-off view is possible, but its proof must appear in the artifact and its construction cost must

count in any comparison. Repeated use can amortize that cost. This is a central distinction between genuine abstraction and moving a long proof to an unseen file.

The same view can support several surface styles. A terse research proof might invoke it by name. A teaching proof might display the correspondence explicitly. The elaborated evidence is the same mathematical object; the presentation is a lens over it.

5 Worked example I: reflection symmetry through a normal form

5.1 The mathematical object

The following generalization isolates the mechanism underlying the reviewed evenness theorem. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be arbitrary and define

$$u_f(x) = \begin{cases} f(x+1), & x \leq 0, \\ f(1-x), & x > 0. \end{cases}$$

For the `ProveIt` example, take $f = \text{fabiusReal } F$. Boundedness and the differential equation are unnecessary for the statement below; the symmetry comes from the folding operation itself.

Proposition 5.1. *For every real x , $u_f(x) = f(1 - |x|)$. Consequently u_f is even.*

Proof. If $x \leq 0$, then $|x| = -x$, so $1 - |x| = 1 + x = x + 1$, and the first branch gives the result. If $x > 0$, then $|x| = x$, so the second branch gives it. Thus

$$u_f(-x) = f(1 - |-x|) = f(1 - |x|) = u_f(x).$$

No property of f was used. □

5.2 Proposed surface proof

```
theorem folded_even (f : Real -> Real): Even (folded f)
proof
  claim normal_form:
    forall x, folded f x = f (1 - abs x)
    by cases (x <= 0); normalize [folded, abs_by_sign]
    conclude by rewrite normal_form; symmetry abs_neg
end
```

Here `abs_by_sign` denotes a registered conditional rewrite package, not a claim that those words are current Lean syntax. The `forall` binder belongs to the claim; the method receives a fresh `x` in its local context. The case split must be justified by the logic profile's treatment of the ordered field.

The elaborator generates a universal normal-form lemma, two branches for its proof, and then an evenness proof using the absolute-value identity. Each branch is a small checked obligation. A reader can inspect the normal-form lemma without seeing every branch simplification.

5.3 What was gained, and what was not

The gain is not principally a reduction from one particular line count to another. It is the identification of a mathematical invariant: the function depends on x only through $|x|$. That invariant is reusable for support statements, regularity away from the fold, and other reflection arguments. The same result could be obtained in ordinary Lean by adding the corresponding helper lemma. **Reason** makes declaring and using such a view the normal interaction pattern.

There is a useful limit to the analogy. The absolute-value normal form alone does not prove differentiability at the folding point. A language must not transfer the smoothness of f across the fold without checking the appropriate compatibility conditions. Conciseness for one theorem must not become an unsound general-purpose rewrite policy.

6 Worked example II: a factorial identity and honest abstraction cost

Define

$$O_n = \prod_{i=0}^{n-1} (2i + 1), \quad O_0 = 1.$$

The arithmetic development proves

$$(2n)! = 2^n n! O_n. \quad (1)$$

Its existing induction proof is already compact. In source notation with the binder type written as `Nat`, its essential structure is:

```
induction n with
| zero => simp [oddDoubleFactorial]
| succ n ih =>
  rw [show 2 * (n + 1) = (2 * n + 1) + 1 by omega,
      Nat.factorial_succ (2 * n + 1),
      Nat.factorial_succ (2 * n), ih,
      pow_succ, Nat.factorial_succ n]
  simp only [oddDoubleFactorial, Finset.prod_range_succ]
  ring
```

This is an attributed excerpt of the proof method, not a complete standalone file [2]. It is important not to caricature this baseline as unintelligible.

6.1 Two mathematical proofs

Inductive proof of (1). At $n = 0$, every factor is 1. If the identity holds at n , then

$$\begin{aligned} (2n + 2)! &= (2n + 2)(2n + 1)(2n)! \\ &= 2(n + 1)(2n + 1) 2^n n! O_n \\ &= 2^{n+1} (n + 1)! O_{n+1}. \end{aligned}$$

The last equality uses $O_{n+1} = O_n(2n + 1)$. □

Partition proof of (1). Partition $\{1, \dots, 2n\}$ into its even and odd members. The maps $i \mapsto 2i$, for $1 \leq i \leq n$, and $i \mapsto 2i + 1$, for $0 \leq i < n$, give disjoint parametrizations of these two parts. Therefore

$$(2n)! = \left(\prod_{i=1}^n 2i \right) \left(\prod_{i=0}^{n-1} (2i + 1) \right) = 2^n n! O_n.$$

The empty products cover $n = 0$. □

The second proof exposes a different human idea: grouping factors by parity. It need not be computationally preferable. The language should permit either style rather than force every proof into a fixed automated normal form.

6.2 A concise induction, with a named method

```

proof
  induct n
  case zero: conclude by normalize
  case succ n with ih:
    expand last factors in factorial and odd_product
    rewrite ih
    conclude by semiring
end

```

The command `expand last factors` is a library method selecting the applicable recurrence theorems at specified occurrences. Its generated obligations include the index equality $2(n+1) = 2n+2$. It is not allowed to unfold arbitrary recursive definitions indefinitely.

A partition-oriented alternative would request `partition_product parity`. The registered parity partition would carry coverage, disjointness, and the two parametrizations. Those proofs are not erased: they move to a reusable library theorem and are counted once in a full cost comparison. If such a library theorem does not exist, the system should either construct and display it or leave the obligation open. Claiming an improvement while omitting that initial cost would be misleading.

7 Worked example III: finite differences and certified reindexing

7.1 The formula and its proof

Let K be a commutative ring, E an additive commutative group, and $f : K \rightarrow E$. For $b, c \in K$, set

$$(\Delta f)(z) = f(bz + c) - f(bz - c).$$

Write $[n] = \{0, \dots, n-1\}$, and define

$$q_{n,T}(z) = b^n z + c \sum_{k=0}^{n-1} b^k - 2c \sum_{k \in T} b^k.$$

The notation $(-1)^{|T|} f(q)$ below means the canonical integer scalar action on E ; it does not assume multiplication on E .

Theorem 7.1. *For every $n \in \mathbb{N}$ and $z \in K$,*

$$(\Delta^n f)(z) = \sum_{T \subseteq [n]} (-1)^{|T|} f(q_{n,T}(z)). \quad (2)$$

Proof. For $n = 0$, the sole subset of $[0]$ is the empty set, both internal sums vanish, and $b^0 = 1$. The right side is $f(z)$.

Assume the formula for n , at every input. Applying it at $bz + c$ and at $bz - c$ gives

$$(\Delta^{n+1} f)(z) = \sum_{T \subseteq [n]} (-1)^{|T|} f(q_{n,T}(bz + c)) - \sum_{T \subseteq [n]} (-1)^{|T|} f(q_{n,T}(bz - c)).$$

For $T \subseteq [n]$, one has $n \notin T$. Direct expansion gives

$$q_{n,T}(bz + c) = q_{n+1,T}(z), \quad (3)$$

$$q_{n,T}(bz - c) = q_{n+1,T \cup \{n\}}(z). \quad (4)$$

For example, the additional term in the geometric sum is cb^n ; including n in the final subset subtracts $2cb^n$, converting that contribution to $-cb^n$.

Every subset of $[n+1]$ either omits n , in which case it is a subset of $[n]$, or contains n , in which case it is uniquely $T \cup \{n\}$ with $T \subseteq [n]$. In the latter case, $|T \cup \{n\}| = |T| + 1$, so the sign is the negative of the sign for T . Substituting (3)–(4) and combining the two disjoint parts proves the induction step. $\square$

7.2 The reusable interface is an equivalence

The appropriate library object is not a heuristic that tries to simplify finite sums until they look similar. It is the equivalence

$$\mathcal{P}([n+1]) \simeq \mathcal{P}([n]) \sqcup \mathcal{P}([n]),$$

whose inverse sends a left element T to T and a right element T to $T \cup \{n\}$. In the forward direction, membership of n determines the summand, and the right branch removes n .

The inverse laws certify uniqueness and eliminate accidental multiplicity. Membership proofs certify that each branch has the stated domain. A cardinality lemma certifies the sign change. An aggregation theorem transfers finite sums along the equivalence. These facts explain exactly why the human instruction “split according to the last index” is sufficient once this interface is available.

7.3 Proposed presentation

```
proof
  induct n generalizing z
  case zero: conclude by normalize
  case succ n with ih:
    expand Delta once
    rewrite ih at (b*z + c), (b*z - c)
    partition target subsets by membership n
      using powerset_last_equiv
    match branches using cube_plus, cube_minus
    conclude by additive_normalization
end
```

The line `generalizing z` is intentionally visible: the induction hypothesis is needed at two new inputs. The line naming the partition records the mathematical idea. The branch matcher receives exact equalities (3) and (4), including their subset hypotheses. The final normalization operates in an additive group and handles integer signs; it must not silently assume a field structure on E .

A typical expanded obligation report would contain the domain check $T \subseteq [n]$, the fact $n \notin T$, the two point identities, the insertion cardinality identity, and the partition equivalence. Some are discharged by registered theorems and some by algebra. The author need not repeat the implementation pattern for every powerset induction, but all of its mathematical content remains in the checked proof.

7.4 Why unrestricted reindexing is not enough

Suppose an author writes “reindex by $T \mapsto T \cup \{n\}$ ” without restricting the domain. That map is not injective on all finite subsets: a set already containing n has the same image as that set with n removed. A reindexing interface must either recover the domain restriction and prove injectivity there, use a theorem that accounts for fibers, or reject the step. This is an example of a failure message that should explain a mathematical obstruction, not invite more arbitrary search.

8 Worked example IV: differentiating a local orbit identity

8.1 The hypotheses are part of the idea

For a real normed vector space E , let $f : \mathbb{R} \rightarrow E$, let $s \subseteq \mathbb{R}$ be open, and fix real a, b, c . Retain the operator Δ from Section 7. Assume

$$z \in s \implies bz + c \in s \quad \text{and} \quad bz - c \in s, \quad (5)$$

and assume that f has derivative

$$f'(z) = a \Delta f(z) \quad (z \in s). \quad (6)$$

Here (6) means an actual derivative assertion at each point, not merely equality involving a totalized derivative operator at possibly nondifferentiable points. This is a real-specialized exposition of the more general normed-field statement in the reviewed file. It preserves the local hypotheses while avoiding unrelated scalar-field notation.

Theorem 8.1. *Under these assumptions, f has derivatives of every finite order on s , and*

$$f^{(n)}(z) = a^n b^{\binom{n}{2}} \Delta^n f(z) \quad (n \in \mathbb{N}, z \in s). \quad (7)$$

8.2 A complete mathematical argument

We first isolate two elementary facts. The operator Δ is linear over constant real scalars. It is also local relative to the invariant set: if $g = h$ on s , then $\Delta^n g = \Delta^n h$ on s . The latter assertion follows by induction because every evaluation at $bz + c$ or $bz - c$ stays in s .

Next, if g is differentiable on s with derivative function g_1 there, the chain rule and (5) show that

$$(\Delta g)'(z) = b g_1(bz + c) - b g_1(bz - c) = b \Delta g_1(z), \quad z \in s.$$

Inducting on the number of applications of Δ , using the chain rule at the two transformed inputs, yields

$$(\Delta^n g)'(z) = b^n \Delta^n g_1(z). \quad (8)$$

This also proves differentiability of $\Delta^n g$ on s ; it does not assume the conclusion as an additional regularity hypothesis. Applying it to $g = f$ and $g_1 = a \Delta f$, and using linearity, gives

$$(\Delta^n f)'(z) = a b^n \Delta^{n+1} f(z). \quad (9)$$

Proof of Theorem 8.1. For $n = 0$, the claim is the identity $f = f$. Suppose the formula holds on all of s for a fixed n , and write $C_n = a^n b^{\binom{n}{2}}$. For $z \in s$, openness gives a neighborhood of z contained in s . On that neighborhood,

$$f^{(n)} = C_n \Delta^n f.$$

Thus the two sides have the same derivative at z . By (9), the right side is differentiable and its derivative is

$$C_n a b^n \Delta^{n+1} f(z).$$

Finally,

$$C_n a b^n = a^{n+1} b^{\binom{n}{2} + n} = a^{n+1} b^{\binom{n+1}{2}}.$$

This proves both existence of the next derivative and the next identity. The induction is simultaneous over every $z \in s$. $\square$

No division by a or b is used. In particular, their vanishing causes no exceptional branch, and the natural-power convention covers the zero-order case. Retaining $\binom{n}{2}$ also avoids an unnecessary natural subtraction in the exponent.

8.3 A language should expose locality, not hide it

```

proof
  induct n generalizing z
  case zero: conclude by normalize
  case succ n with ih:
    claim local_identity:
      nth_derivative n f = C(n) * Delta^[n] f near z
      from ih, hs, hz by restrict_to_neighborhood
    differentiate local_identity at z
      using affine_orbit_derivative, invariant_plus,
        invariant_minus, differential_equation
    conclude using choose_succ_two by scalar_normalization
end

```

The notation `near z` elaborates to equality on a neighborhood filter, or to a certified equivalent local formulation. It is not a comment. The differentiation method requires an equality in that form and derivative evidence for the expressions it differentiates.

A tempting but invalid surface rule would be “from $p(z) = q(z)$, differentiate both sides.” For example, $p(x) = x$ and $q(x) = 0$ agree at zero but have different derivatives there. The correct abstraction compresses the proof that a local identity exists; it does not compress away the locality requirement.

The induction generalization is likewise substantive. An induction hypothesis only at the original point would not provide the identity on a neighborhood. The surface syntax deliberately keeps this choice visible. An assistant may recommend generalizing z , but must not silently change the induction statement after presenting it as fixed.

9 Worked example V: integer square roots by double counting

The previous examples reorganized existing proof mechanisms. This example also changes the mathematical proof strategy. It illustrates how a language should support a human-selected intermediate object rather than merely search harder for a tactic chain matching the existing implementation.

Let $n \in \mathbb{N}$, set $s = \lfloor \sqrt{n} \rfloor$, and write

$$S(n) = \sum_{k=1}^n \lfloor \sqrt{k} \rfloor.$$

The target is

$$S(n) = s(n+1) - \frac{s(s+1)(2s+1)}{6}. \quad (10)$$

We will prove it in a way that also explains why the quotient is exact and the natural subtraction does not truncate.

9.1 The finite relation

Define the finite set

$$D = \{(j, k) \in \{1, \dots, s\} \times \{1, \dots, n\} : j^2 \leq k\}.$$

Fixing k with $1 \leq k \leq n$, the permitted positive j 's are exactly $1, \dots, \lfloor \sqrt{k} \rfloor$. Indeed, for natural j , $j^2 \leq k$ is equivalent to $j \leq \lfloor \sqrt{k} \rfloor$, and monotonicity of the integer square root ensures that this bound is at most s . Hence

$$|D| = \sum_{k=1}^n \lfloor \sqrt{k} \rfloor = S(n). \quad (11)$$

Fixing j with $1 \leq j \leq s$, one has $j^2 \leq s^2 \leq n$. The permitted k 's are exactly the interval $j^2, \dots, n$, containing $n + 1 - j^2$ elements. Therefore

$$|D| = \sum_{j=1}^s (n + 1 - j^2). \quad (12)$$

Let $Q = \sum_{j=1}^s j^2$. Because $j^2 \leq n + 1$ for each index, ordinary distributivity together with these bounds gives the natural-number identity

$$|D| + Q = s(n + 1). \quad (13)$$

In particular $Q \leq s(n + 1)$. This additive balance is often a better intermediate statement than a subtraction formula.

9.2 Exact division, without an unsafe cast

For completeness, the sum-of-squares identity in a division-free form is

$$6Q = s(s + 1)(2s + 1). \quad (14)$$

At $s = 0$, both sides are zero. Increasing s to $s + 1$ increases the left side by $6(s + 1)^2$. The difference of the right sides is

$$(s + 1)(s + 2)(2s + 3) - s(s + 1)(2s + 1) = 6(s + 1)^2,$$

by multiplication and collection of terms. Induction proves (14).

It follows that 6 divides the numerator in (10), and its natural quotient is exactly Q . Equation (13) implies $S(n) = s(n + 1) - Q$, with no truncation. Combining these statements proves (10). All sets and sums are empty when $n = 0$, so that case is included.

9.3 Proposed presentation and its obligations

```
proof
  let s := isqrt n
  let D := {(j,k) in Icc(1,s) * Icc(1,n) | j^2 <= k}
  count D by k using isqrt_characterization
  count D by j using interval_cardinality
  claim balance:
    sum(k in Icc(1,n), isqrt k)
    + sum(j in Icc(1,s), j^2) = s*(n+1)
  from the two counts by finite_sum_normalization
  conclude from balance, sum_squares_times_six
    by exact_natural_arithmetic
end
```

The two count commands are library methods for finite fibers, not informal appeals to a counting oracle. Their output is a cardinality equality with a proof. The second method must establish $j^2 \leq n$ before using the interval cardinality. The final method proves the division and subtraction obligations from the additive balance and the cross-multiplied square-sum formula.

The existing rational-induction route is a legitimate alternative. A certified Nat-to-Rat view should support it. The double-counting route shows an additional design goal: the system should let an author introduce a useful relation and reason about it at its natural mathematical level. This may improve explanation more than replacing several casts with a single command.

10 A small core semantics

The following semantics describes the intended evidence discipline. It is not a mechanized metatheory of a completed implementation. The objective is to identify obligations that an implementation must satisfy and to show why its checking boundary is adequate under explicit assumptions.

10.1 Fixed targets and local contexts

Write

$$\mathcal{E}; \Gamma \vdash p : G$$

for the underlying Lean typing judgment, with $G : \text{Prop}$. A local context is an ordered telescope of typed variables, local definitions, and hypotheses. Order matters because later types can depend on earlier variables.

Before search begins, the theorem statement and its externally visible parameters must be fully elaborated. In particular, the target must not contain unresolved metavariables that search is allowed to instantiate to a different proposition. Proof construction may use internal metavariables, but they must be resolved before a candidate is accepted. The same requirement applies to universe metavariables and hidden instance choices relevant to the frozen statement.

A proof-plan state is a finite collection of scoped obligations. An obligation records an exact target, a local telescope, earlier accessible claims, and a reconstruction contract. A claim cannot refer to itself, a future sibling claim, or a local witness that has left scope. Recursive mathematical reasoning is represented through a checked induction or well-founded recursion principle, not through a cycle in the claim graph.

10.2 Core elaboration rules

The judgment

$$\mathcal{E}; \Gamma \vdash s \Downarrow p : G$$

means that source proof s elaborates to evidence p for the fixed goal G . Some representative rules are as follows.

Assumption. If $h : G$ is accessible in Γ , an explicit reference to h elaborates to h . Accessibility includes lexical scope and any restrictive premise policy in force.

Implication introduction. To prove $A \rightarrow B$, a block `assume h : A` followed by a proof of B elaborates to $\lambda h : A. p$. Introducing this hypothesis is allowed because the fixed target is an implication. A surface command cannot silently prepend an arbitrary premise to another target.

A named claim. A block that proves A as h , then proves G using h , elaborates according to

$$\frac{\mathcal{E}; \Gamma \vdash p : A \quad \mathcal{E}; \Gamma, h : A \vdash q : G}{\mathcal{E}; \Gamma \vdash (\lambda h : A. q) p : G}.$$

The proof p is constructed without h in scope. This simple requirement excludes the circular error of treating the intended claim as an assumption while proving it.

Reduction of the goal. A suffices `B via r` step requires a checked bridge $r : B \rightarrow G$ in the current context. A proof $p : B$ then closes the original goal with $r p$. A generated bridge can itself have sub-obligations, but none may be discarded. Without a bridge, replacing a goal by a convenient stronger or unrelated statement is not justified.

Existential elimination. Given $e : \exists x : T, P(x)$, a scoped obtain x, h using e introduces fresh $x : T$ and $h : P(x)$ to prove a proposition G independent of the fresh witness. It elaborates through the existential eliminator. The freshness and scope conditions prevent x from escaping in G or in previously declared data.

Cases and induction. A case split requires a checked exhaustive eliminator for the scrutinee. Induction requires an explicit recursor, motive, and generalized parameters. Surface conveniences may reconstruct these objects, but the accepted result is an application of the underlying eliminator with every branch checked at its instantiated type.

Equality transport. A replacement of u by v requires $e : u = v$, or a suitable registered relation with a congruence theorem. For a dependent target $P(u)$, the result is transported along e . Simple syntactic rewriting is not an adequate specification of dependent transport.

Reconstruction. Any permitted search procedure may return a candidate p . The only acceptance rule is

$$\frac{\text{Check}_{\mathcal{E}}(\Gamma, p, G) = \text{accept} \quad \text{PolicyOK}(\mathcal{C}, p)}{\mathcal{E}; \Gamma \vdash \text{reconstruct } \mathcal{C} \Downarrow p : G}.$$

The first premise establishes typing. The second checks restrictions such as permitted axioms and dependencies. Neither a search score nor a success flag substitutes for the first premise.

10.3 Relative soundness

Theorem 10.1 (Relative evidence soundness). *Assume that the target and environment are fixed; every local elaboration rule produces the underlying typed term described above; every reconstructed leaf is synchronously checked at its original contextual target; the obligation graph is finite, acyclic, and scope-correct; and the final assembled declaration is checked again. Then acceptance of a closed Reason proof of G produces a term p such that $\mathcal{E}; \emptyset \vdash p : G$.*

Proof. Topologically order the proof obligations. A leaf is either an accessible assumption, a checked library term, or a reconstructed term accepted by the checker, so it has its recorded type. For each subsequent node, assume the terms for its prerequisites have the recorded contextual types. The implication, claim, reduction, eliminator, and transport rules combine these terms through typing-preserving constructions. Freshness and scope checks make the required substitutions valid. Thus that node has its recorded type. Induction over the finite order yields a term for the root. The final declaration check independently validates the fully assembled term at the frozen closed target, giving the stated judgment. $\square$

This theorem is intentionally relative. It does not prove Lean’s consistency, correctness of the checker implementation, or agreement between an informal intention and the frozen proposition. Those are distinct trust questions. Nor does the theorem establish that an implementation of the proposed parser obeys these rules. A production system should formalize as much of the front-end elaboration interface as is practical, and should rely on independent final checking rather than only on the argument above.

10.4 Two useful consequences

Corollary 10.2 (Search independence). *Replacing a reconstruction engine by an arbitrary candidate generator preserves relative evidence soundness, provided the same acceptance checks and fixed target are retained.*

Proof. The reconstruction rule depends on a checked candidate, not on the algorithm that found it. Rejected, malformed, or absent candidates establish no judgment. $\square$

Corollary 10.3 (Safe failure). *A timeout or exhausted search budget cannot establish the requested proposition, unless a separately completed checked proof has already been retained.*

Proof. Neither timeout nor exhaustion is a premise of an evidence-producing rule. A previously checked proof, when present, supplies evidence independently of the later failure. $\square$

10.5 Existence proofs are not automatically programs

Lean’s distinction between propositions and computational data must remain visible. Destructing a proof of $\exists x : T, P(x)$ inside a proof of another proposition is not the same operation as extracting executable data of type T . A computational witness should normally be provided as a dependent pair with the appropriate data-level eliminator. A classical choice operation is a different interface with an explicit logical dependency [14].

Consequently choose must not be an overloaded escape hatch that sometimes means existential elimination and sometimes means unannounced classical extraction. The editor should show whether a witness is local proof data, a constructive returned value, or a value obtained under a classical policy. Different inhabitants of a data type can behave differently even when alternative proof terms of the same proposition are interchangeable for the theorem’s truth.

11 Ambiguity, intent, and the limits of inference

11.1 Statement elaboration and proof search are different phases

Consider “the square root,” “division by six,” or “the Fabius function.” In the examples, plausible meanings include a natural integer square root, a real principal square root, natural division, rational division, a bounded cumulative function, and a signed extension. Selecting among these is not a mere tactic choice.

Reason should first elaborate the theorem interface using ordinary type information and explicit project conventions. Deterministic, unambiguous inference is useful: omitted implicit arguments and routine typeclass instances should remain inferable. But when two materially different interpretations survive, the system must request a disambiguation or show the choice for explicit commitment before proof search. It must not select whichever statement happens to be easiest to prove.

A practical statement lock records the elaborated type, universe parameters, binder order, selected definitions, and relevant instances. A human-readable expansion displays the locked meaning. Proof search may propose several proofs of this object; changing the object is a separate edit that invalidates its certificates.

11.2 Proof ambiguity is usually benign; explanation ambiguity is not

If two terms prove the same fixed proposition under the same permitted assumptions, either establishes the theorem. No uniqueness of proof search is required. Nevertheless, they can have different axiom dependencies, performance, stability, or explanatory value. Those differences justify ranking and policy checks, not a claim that only one proof is mathematically legitimate.

A line saying “by monotonicity” raises an additional question. A search procedure could find a correct proof using an unrelated strong theorem. That establishes the target, but does not automatically justify the displayed explanation. There should therefore be two annotations: a permissive by `auto` that

promises only checked evidence, and a method-constrained annotation whose accepted reconstruction must use a registered monotonicity schema or a transparently equivalent derivation.

Even this is not a solution to the philosophy of explanation. A dependency appearing syntactically in a proof term is not necessarily causally necessary, and a minimal dependency set is not generally easy to compute. The system should report “used in the elaborated proof” or “checked instance of this schema,” not claim that it has discovered the unique reason the theorem is true.

11.3 Locality and mathematical focus

A theorem search service may find a library theorem that is exactly the goal. That is valuable when the author wants a citation. It may be unhelpful when the author is trying to expose an induction argument. Contracts should therefore support an allowed method set and a preferred local premise set, with an explicit opt-in widening step.

The contract must also distinguish *ranking preferences* from *hard dependency restrictions*. A preferred premise is not a guarantee that no other declaration will be used. A hard restriction requires checking the generated term and its relevant transitive dependencies against the policy. Ordinary logical infrastructure and theorems encapsulated by a registered method need a documented treatment, rather than pretending that the final proof depends only on the two labels visible in a sentence.

11.4 “Without loss of generality” needs a theorem

For a goal $\forall x : X, P(x)$, a valid reduction to a subset $S \subseteq X$ can be packaged by proving that every x has a suitable representative $r(x) \in S$, and that $P(r(x))$ implies $P(x)$. A symmetry action is one common way to obtain these facts, but not the only one.

A wlog x in S using reduction command should elaborate to this explicit bridge and generate its side conditions. It must not merely create the assumption $x \in S$. For example, exchanging two variables in a symmetric inequality requires invariance of both the assumptions and the conclusion. Preserving only the conclusion is insufficient when the hypotheses distinguish the variables.

11.5 An inconsistent context is not a successful explanation

A proof from a contradictory local context can be logically correct as an implication. It may nevertheless reveal a mistaken theorem statement or an erroneous earlier assumption. A production interface should flag a contradiction when one is discovered, and make uses of elimination from falsehood visible. It cannot promise to detect all inconsistent contexts. The desired policy is transparency, not an unsound attempt to reject every theorem whose assumptions are hard to satisfy.

12 Synthesis architecture inspired by Leant

12.1 A layered reconstruction pipeline

The proposed pipeline begins with cheap, predictable operations and broadens only when the contract permits. First, elaborate the local mathematical expressions and settle definitional equalities. Second, perform a small set of certified normalizations. Third, run structural synthesis over the local implication, conjunction, product, or existential interfaces that the chosen fragment actually supports. Fourth, retrieve nearby library facts and invoke domain methods. Finally, use broader heuristic or model-assisted search when explicitly enabled.

This order is a policy, not a logical requirement. A domain-specific arithmetic solver may be the cheapest first step for one project. What matters is that each stage has the same exact target and

returns evidence in a common checked format. A later stage does not inherit permission to change assumptions simply because earlier stages failed.

Leant’s structural synthesis offers an especially useful middle ground between manual term construction and unconstrained search. The search engine can discover connective structure and argument wiring while treating difficult dependent subexpressions as opaque. The original target remains the final authority. Such opacity may reduce completeness; it need not reduce soundness when checking is performed at the original target.

12.2 Search representation versus proof representation

A neutral search representation can make sharing, canonicalization, candidate ranking, and fragment analysis tractable. It should preserve exact identity for atoms and the dependency information needed for binders. Two superficially similar atoms, such as $P(n)$ and $P(n + 1)$, must not collapse into the same symbol merely because they have the same outer predicate name.

A candidate in that representation is not yet a Lean proof. Lowering must restore the original constants, indices, binders, coercions, and universe information. Candidate construction can use metavariables internally, but checking accepts only a closed contextual expression with no unresolved term or universe holes. The Lean-native architecture discussed in Leant’s rewrite study is a relevant precursor here [12].

The design should not require the search engine to use raw Lean expressions for every internal operation. Nor should it require every candidate to make a round trip through pretty-printed source. A worker owning the exact environment can lower to native expressions and check them directly; printing is then a presentation operation with its own copy-and-paste validation if that feature is offered.

12.3 Candidate ranking is a usability mechanism

For candidates proving the same target, reasonable preferences include fewer gratuitous case splits, fewer unrelated lemmas, lower expected checking cost, and preservation of a user-specified mathematical schema. A short term is not always the best term. Inlining a shared subproof can shorten one displayed line while increasing both checking cost and reader confusion.

Candidate diversity is also useful. Two genuinely different strategies may teach more than ten variants differing only in argument order. Leant’s candidate-quality work is relevant because it treats ranking and normalization as bounded-search concerns rather than correctness certificates [9].

A proposed ranking tuple might prioritize policy compliance, schema fidelity, successful checking, and then an estimate of replay and explanation cost. In practice, candidates can be checked in ranked batches to avoid checking an enormous pool. This introduces a latency/quality tradeoff that should be measured rather than hidden behind a claim of optimal proof selection.

12.4 Outcome types must retain distinctions

A useful result interface distinguishes at least the following cases:

```
CheckedProof(target, environment, term, dependencies)
CheckedNegation(target, environment, term)
CheckedCounterexample(statement, witness, evidence)
FragmentDiagnostic(fragment, translation, result)
CandidateRejected(reason)
BudgetExhausted(resources_used)
Unsupported(feature)
Cancelled
```

A checked negation contains a proof of the negation at its own exact target. A checked counterexample to a universally quantified claim contains an actual witness together with appropriate mathematical evidence. A fragment diagnostic describes a fact about the abstraction used in search. The remaining outcomes do not assert a mathematical theorem.

Even an exhaustive failure in a sound propositional proof-search fragment does not generally yield a proof of the negated goal. With an arbitrary proposition P and no assumptions, failing to derive P does not establish $\neg P$. Unprovability, unsatisfiability, absence of an inhabitant in a specified calculus, and a proof of negation are different statements. A complete solver can justify precisely the metatheoretic conclusion covered by its specification, and no more.

This distinction is particularly important when unsupported dependent structure has been abstracted as atoms. A negative conclusion about the abstraction may not transfer to the original environment at all. The front end should retain the translation's completeness and losslessness information, and display a bounded miss as a bounded miss.

12.5 Behavioral hints are not proof certificates

A synthesized function may satisfy a shape or length heuristic without satisfying its requested theorem. An SMT model may describe the solver's abstract semantics rather than the concrete Lean function. Testing a candidate on a finite set of inputs may be useful for prioritization, but cannot establish a universal property.

The appropriate integration is asymmetrical. Heuristics may help choose what to check next. A concrete counterexample can justify rejection when its interpretation is validated. A positive theorem is accepted only through the designated proof-checking path. A solver's success flag, a callback-acceptance wrapper, and a kernel-checked proof must have different types or states in the implementation.

12.6 Incrementality and worker ownership

The proof worker should own the elaborated context and the native expressions attached to it. A request carries a worker generation, a context identity, an exact target identity, and the reconstruction contract. A response from an older worker generation is stale even when its printed goal looks identical.

Cancellation must stop resource consumption without converting partial work into evidence. A checked candidate obtained before cancellation can be retained, provided its origin is still valid. Candidate IDs and opaque capabilities should not be serialized as if they were durable proof certificates. Reopening a project requires rebuilding the exact environment and checking exported evidence, not reviving a token from a previous process.

13 Trust, publication, and reproducibility

13.1 Two validation problems

There are two independent questions. Does the evidence prove the elaborated theorem? Does the elaborated theorem express the author's intended mathematics? A kernel addresses the first relative to its environment and trusted mechanisms. It cannot decide the second merely from a polished natural-language description.

A publication workflow should therefore retain both the exact theorem interface and the checked proof. The interface is compared with the previously approved statement, rather than being supplied only by the same untrusted generator that supplies the proof. This separation is also relevant to adversarial tactics: changing a definition or adding a convenient axiom can make an irrelevant theorem easy to prove.

Lean’s validation documentation discusses axiom auditing, separate checking, and adversarial execution contexts. It also distinguishes ordinary proof checking from native-evaluation trust. A production policy must be version-aware: inspecting only a historical compiler-trust marker is not a reliable substitute for examining the actual axiom dependencies of the selected toolchain [14].

13.2 Axiom policy is part of the theorem’s context

Different projects may permit classical choice, propositional extensionality, quotient principles, additional mathematical assumptions, or native-computation certificates with an explicitly enlarged trusted base. The language should not label all such policies simply “verified” and hide their differences.

A useful export record includes the transitive axiom dependencies, with an explicit allowlist. Unfinished placeholders such as `sorry` and unapproved custom axioms must fail a publication policy rather than being treated as successful reconstructions. Conditional mathematical results are perfectly legitimate, but their assumptions belong in the approved theorem interface or an openly identified environment policy.

The same caution applies to proof-producing decision procedures. A small verified checker for a certificate has a different trust profile from accepting an external solver’s Boolean answer. Both may be useful engineering tools, but the record must state which one established the result.

13.3 Search-free replay

Exploration may use nondeterministic search, a remote language model, or a large premise-retrieval index. Publication should not require these services. The artifact should contain a resolved proof representation or an ordinary Lean proof source that deterministically reconstructs the same checked evidence without rediscovering its strategy.

There are several engineering choices. A fully explicit Lean term can be large. A deterministic proof script can be smaller but depends on elaborator and tactic versions. A native serialized proof object may replay efficiently but has format and toolchain constraints. The design need not prematurely choose one universal encoding. It should define the validation contract for each encoding and ship the environment needed to check it.

A durable record might include

<code>statement:</code>	exact elaborated theorem interface
<code>proof:</code>	resolved term or deterministic replay source
<code>environment:</code>	toolchain + dependency lock + declaration identities
<code>policy:</code>	permitted axioms and validation mode
<code>provenance:</code>	source spans, methods, actual dependencies
<code>validation:</code>	checker configuration and completion result

Hashes are useful lookup keys and corruption checks; they are not mathematical evidence. A digest matching an old result is not enough to accept a proof without the validation policy’s required recheck. Similarly, a cached typechecked callback result is not an everlasting authority after the environment changes.

13.4 A practical three-mode interaction

Explore mode permits broad search and unfinished claims, displays conjectural suggestions as suggestions, and never exports unfinished results as established theorems. *Resolve mode* replaces successful reconstructions with checked candidates, records their dependencies, and makes pending obligations explicit. *Publish mode* disables search, rejects forbidden assumptions and holes, checks the approved statement, and validates the resolved evidence.

These modes separate responsiveness from durability. An author need not use publication-level restrictions while experimenting, but the interface should never make an exploratory suggestion look indistinguishable from a published proof. A proof can be mathematically complete even if its explanation still needs editing; the display should distinguish those states too.

13.5 Security and privacy

A metaprogram can execute arbitrary host-side actions even when its generated proof is later rejected. Logical soundness is not a sandbox. Untrusted automation should run with resource limits, restricted filesystem access, and an execution policy appropriate to the threat model. Independently checking the output does not undo a file deletion or prevent an unwanted network request [14].

Remote premise or model services introduce a separate confidentiality question. Sending an unpublished theorem, local definitions, or proprietary context should be an explicit project choice. A useful architecture can keep exact contexts local, send a minimized query when authorized, and check every returned candidate locally. None of these privacy decisions should alter what counts as proof evidence.

14 The reader’s interface: proof lenses and mathematical diagnostics

14.1 Three synchronized views

The default *mathematical view* shows theorem statements, named claims, essential hypotheses, and the chosen strategy. An expandable *obligation view* shows exact contexts, side conditions, dependencies, and the status of each reconstruction. A *kernel view* shows the generated Lean term or replay source, selected constants and instances, and the axiom audit.

These should be synchronized views of one accepted artifact, not three separately generated stories. Clicking a displayed sentence should identify the exact proposition or proof-schema application that supports it. A generated prose explanation that cannot be linked to such evidence should be marked as commentary, not presented as another certificate.

14.2 Diagnostics should name the mathematical obstruction

For the derivative example, a helpful failure says: “The available equality holds only at the point. This method requires equality on a neighborhood.” For the natural-division example, it says: “This transport requires exact divisibility by six.” For the powerset example, it says: “Insertion is not injective on this domain; the missing condition is that the inserted index is absent.”

Each message should show the unresolved formal obligation and the relevant source span. It may offer a candidate lemma or a suggested generalization, but these remain edits for the author to accept. The system must distinguish a genuine counterexample from its own inability to discover a proof.

14.3 Suggestions should not rewrite accepted intent silently

A suggestion can close a local goal, recommend an intermediate lemma, or propose an alternative proof strategy. It should not automatically replace a pedagogically useful induction by an opaque global theorem, or mutate the theorem’s binder types to make an existing tactic succeed. The advisory character of Leant’s tactic-suggestion workflow is a useful precedent for this interaction boundary [7].

The same rule applies to maintenance. After a library upgrade, a repair tool may find a new proof of the same locked target. If a definition’s meaning or the theorem statement itself has changed, the repair must be presented as a semantic change rather than a harmless proof update.

14.4 Proof boundaries should be stable editing boundaries

A named claim is a natural cache and repair unit. Editing one claim should invalidate its dependents, not necessarily the entire document. Scope and environment changes may force broader invalidation; the interface should explain why. This gives a practical benefit to mathematical structure: a good proof outline is also a useful incremental dependency graph.

Overly fine-grained claims can become tedious, while overly coarse ones conceal the argument and concentrate search cost. The editor can suggest splitting an expensive reconstruction at a discovered invariant or grouping several deterministic normalizations. Such suggestions optimize a readable proof plan, not just line count.

15 Relationship to existing proof languages and tools

15.1 Declarative proof languages are the starting point

Isabelle/Isar demonstrates structured proofs with local assumptions, explicit intermediate statements, and compositional treatment of induction. Its proof language is an important precedent for organizing the human argument rather than merely recording tactic execution [16]. Mizar likewise exemplifies a long-running declarative formalization tradition; a proposal for concise mathematical proofs should not present declarative structure itself as novel [17].

Isabelle/Naproche explores controlled mathematical language, including a LaTeX-oriented surface and automated checking of generated obligations. It is especially relevant to the division between mathematical text and the internal logical tasks it induces. Its architecture and logical setting should not be conflated with Lean’s proof-term checking model [18].

Verbose Lean provides controlled-natural-language tactics aimed at making formal proofs transfer more readily to conventional paper proofs. That goal differs from simply minimizing keystrokes, and aligns with this article’s emphasis on explanatory structure [19].

15.2 Lean already offers much of the foundation

Lean already has declarative intermediate claims, calculation chains, extensible syntax, and proof-producing tactics. Aesop supplies extensible search with normalization and different search-rule classes; its use of “safe” and “unsafe” concerns search behavior, not permission to accept logically unsound conclusions [20]. Current Lean developments also include expanding tactic-suggestion capabilities, so a design should not compare itself only with the automation available in older releases [15].

Thus the recommended implementation is not a new theorem prover with a new library ecosystem. It is a coordinated layer using Lean’s existing strengths. An ordinary Lean expert can often emulate an individual **Reason** example with a helper lemma and a custom tactic. The proposed contribution is to standardize the contract, evidence identity, explanation link, and replay interface across such examples.

15.3 What is specifically added to the Leant direction

Leant addresses candidate discovery and contextual suggestion, and its native rewrite analysis already addresses important expression-checking and worker boundaries. **Reason** adds a language-level unit in which these capabilities are invoked: a stable mathematical claim with an explicit scope and a reconstruction contract. It also gives certified representation views a first-class role, separates statement commitment from proof search, and treats readable publication as a projection of checked evidence.

These are design contributions rather than claims of priority. The value of the combination should be demonstrated through implementations and user studies. It would be a mistake to infer that renaming

an existing tactic interface produces a meaningful new language.

15.4 Why not simply use unrestricted natural language?

Unrestricted prose is valuable for brainstorming and explanations, but has unstable implicit quantifiers, domains, and references. A model may suggest formalizations from prose; the result should then be displayed as a precise candidate statement for commitment. Proof search follows that commitment, rather than validating the model's unstated interpretation retroactively.

A controlled layer can still feel mathematical. It can support conventional notation, local definitions, forward reasoning, nested arguments, and named methods. The restriction is not that every sentence must look like a machine instruction. It is that every proof-relevant construct has a precise elaboration contract, and every ambiguous interpretation is settled before its evidence is accepted.

16 Implementation strategy and migration

16.1 A small embedded prototype first

The first implementation should be a Lean package adding a proof-block syntax, not a standalone parser with its own imitation of the Lean environment. It can initially support `claim`, `from`, `suffices`, `scoped obtain`, `explicit induction`, and a registry of named reconstruction methods. Mathematical expressions should use Lean's elaboration machinery in the project's actual environment.

Each command should produce a typed obligation object and a source-location record. Existing tactics can discharge obligations while the prototype establishes the evidence and status protocol. A Leant bridge can then add structural synthesis without redesigning the language core. The first goal is to make wrong targets, stale contexts, and hidden assumptions impossible to mistake for accepted evidence.

16.2 Milestones with acceptance gates

Milestone A: scoped claims and frozen targets. Implement the core proof blocks, exact target capture, and separate candidate acceptance. Acceptance tests must reject self-reference, escaped witnesses, unresolved holes, and a proof of a different target. A minimal ordinary Lean export must pass the chosen final checker.

Milestone B: reconstruction portfolios. Connect a small deterministic tactic collection and structural synthesis. Enforce resource accounting and distinguish every result class in Section 12.4. Test cancellation, duplicate candidates, rejected candidates, and stale worker replies. Search exhaustion must never produce a negation theorem.

Milestone C: three certified mathematical views. Implement a fold/absolute-value view, a finite-index equivalence view, and exact natural-to-field transport. These directly exercise the repository examples while remaining small enough to audit. Tests must include missing bounds, noninjective maps, and invalid square-root branch simplifications.

Milestone D: explanation and replay. Link each visible proof step to its checked proposition or schema application. Export a search-free artifact with an axiom audit and environment lock. Replaying must work with the synthesis service unavailable. Corrupting or changing a dependency must trigger rechecking or failure.

Milestone E: realistic mathematical use. Port the five worked examples alongside ordinary Lean versions, then expand to additional domains. Measure authoring and repair effort, not just printed size. Only after these results should the syntax and default search policies be treated as stable.

These are dependency-ordered engineering gates, not time estimates or promises about achievable performance.

16.3 Gradual adoption

A development should be able to mix ordinary Lean and **Reason** blocks. A proof can begin as a Lean tactic script, acquire named mathematical claims, and replace repeated implementation patterns by certified methods one at a time. Conversely, an expanded **Reason** proof should be inspectable or exported into Lean for debugging and long-term maintenance.

This avoids a risky all-or-nothing migration. It also provides an honest baseline: if a proposed construct adds complexity without improving comprehension or repair, the author can retain the simpler Lean proof. Language design should accommodate that outcome rather than declare every existing proof obsolete.

16.4 Avoiding an oversized trusted front end

The parser, retriever, model, ranker, and pretty printer can remain outside the logical trust boundary provided the approved statement and generated evidence are independently validated. However, they remain relevant to user intent, execution security, and usability. “Untrusted” is not a synonym for “unimportant.”

An implementation should document these different trust roles explicitly. A small proof-checking boundary supports logical assurance; typed APIs support engineering discipline; sandboxing supports execution safety; a statement lock and transparent notation support semantic accountability. No one mechanism substitutes for the others.

17 Evaluation: what would count as success?

17.1 A fair experimental design

A useful evaluation would select theorems before optimizing their **Reason** versions and stratify them by proof shape: structural logic, algebra, finite sums, local analysis, dependent indices, and representation transport. The five examples in this article are seed tasks, not a statistically representative benchmark.

Compare at least three conditions: the repository proof at a pinned revision; an idiomatic Lean version improved by an experienced author with access to ordinary helper lemmas; and the proposed language using the same mathematical library. Shared helper development should be recorded separately, then amortized under an explicit reuse model. Comparing a mature domain-specific view against an unrefactored one-off Lean proof alone would overstate the language’s advantage.

All conditions should use identical theorem statements and permitted assumptions. Failed formalizations and repairs belong in the results. Selecting only theorems for which the new syntax looks attractive would answer a much weaker question.

17.2 Metrics and interpretation

Report distributions rather than only means: a language with excellent typical latency but frequent very expensive reconstructions may be frustrating in practice. Human studies should distinguish experienced formalizers from mathematicians new to Lean, and should counterbalance task order. A

Metric	What to report
Author input	Tokens and editing actions, with shared helpers counted separately.
Comprehension	Accuracy and effort in explaining the invariant, hypotheses, and a deliberately inserted error.
Reconstruction	First checked candidate latency, total work, failure rate, and resource-limit outcomes.
Checking	Final proof size and independent validation cost, distinct from search.
Repair	Work after renamed lemmas, changed normal forms, extra assumptions, and library upgrades.
Reproducibility	Offline replay success with search disabled and dependencies pinned.
Semantic integrity	Rejection of altered targets, stale contexts, illicit axioms, and missing side conditions.

Table 1: An evaluation protocol, not measured results.

reader’s confidence is not a substitute for their ability to identify the exact claim and its necessary assumptions.

17.3 Ablations

To discover what actually helps, disable the natural-language surface while retaining contracts; disable synthesis while retaining views; disable views while retaining synthesis; and compare broad search with local method-constrained search. These experiments separate benefits from syntax, library abstraction, and search quality.

Another useful ablation compares a named mathematical claim with an opaque one-line automation call that closes the same theorem. If the named-claim version helps readers diagnose a changed hypothesis or repair a proof, it has value even if it is longer. If it only adds ceremonial text, the design should be simplified.

17.4 Adversarial mathematical tests

The benchmark should include intentionally invalid requests: differentiating an equality at one point; transporting natural subtraction without a bound; treating a noninjective index map as a bijection; erasing a square-root absolute value; eliminating an existential witness into unrestricted computational data; and using a candidate from a changed environment.

These cases are not peripheral security exercises. They lie exactly at the boundary between the details mathematicians often omit and the details that make the argument true. A language that appears concise because it mishandles these examples has failed its central objective.

18 Executable companion: a deliberately small evidence model

The archive includes a Python program, `supplement/toy_checker.py`, and its tests. The program implements a typed term checker for atomic types, function types, and binary products. Its terms are variables, typed lambdas, applications, pairs, and projections. It also assembles an acyclic sequence of named claims through lambda applications and implements bounded candidate selection with explicit result statuses.

This is not a Lean kernel, a dependent type theory, or an implementation of the proposed language. It has no quantifiers, inductive families, rewriting engine, theorem retrieval, mathematical views, or natural-language parser. Its purpose is narrower: make the contract between candidate discovery and evidence checking executable and inspectable without an installed Lean toolchain.

The tests exercise valid structural proofs, type errors, free-variable and scope errors, self-reference

and forward reference, exact-origin replay, forged certificate contents, and resource-bound behavior. In particular, a zero candidate budget does not inspect a candidate stream; rejecting one candidate does not validate the next unexamined candidate; and a miss has no negation-proof constructor.

The companion's **32 tests passed** in the generation environment. This is a test result for the toy implementation only, not a formal proof of its correctness, a security audit of Python, or evidence that the full **Reason** design has been implemented. The complete test transcript is included in the archive. No Lean executable was available in this session, and neither repository was compiled.

The toy checker also makes a useful engineering point: its Python certificate constructor is public, so replay does not trust the wrapper. It checks the exact origin and rechecks the stored term. Production systems can strengthen encapsulation, but they still need to identify which operation supplies actual evidence rather than relying on an impressive type name.

19 Risks, open design questions, and conclusions

19.1 The principal risks

The greatest usability risk is a language that looks like ordinary mathematical prose but has surprising hidden rules. The response is a small core, explicit notation commitment, transparent obligations, and consistent method contracts. Natural-looking syntax alone is not enough.

The greatest engineering risk is uncontrolled reconstruction cost. Locality, bounded portfolios, incremental checking, and search-free replay mitigate it, but cannot guarantee that every plausible mathematical sentence has an inexpensive proof. Expensive steps should be split into mathematical lemmas, not hidden behind a larger timeout.

The greatest evaluation risk is measuring the wrong thing. A proof can become shorter because a thousand-line helper was excluded from the count, because a theorem was weakened, or because the explanation disappeared into a global automation call. A useful evaluation must expose all three possibilities.

Finally, excessive automation may reduce the author's awareness of assumptions. Proof lenses and side-condition summaries should counteract this, but the best interaction design remains an empirical question. Some projects may reasonably choose stricter defaults that display every nontrivial transport precondition.

19.2 Research questions worth pursuing

One question is how to infer an effective reconstruction contract from an accepted proof: which small set of methods and premises reproduces the argument without overfitting a particular library version? Another is how to identify a stable mathematical view across different formal representations. A third is how to rank proof plans by explanatory quality without equating readability with syntactic smallness.

There is also a promising bidirectional problem. Given a checked Lean proof, can the system recover a hierarchy of mathematical claims and reusable schemas, then verify that its reconstructed explanation faithfully describes those schema instances? This need not solve unrestricted mathematical understanding to be useful. Even reliable recognition of parity partitions, finite-fiber counts, and neighborhood-restriction arguments would support the examples here.

These questions call for targeted prototypes and evidence. The existence of a sound final checker permits experimentation with aggressive, imperfect proposal mechanisms; it does not by itself answer the interface or explanation questions.

19.3 The central design principle

The proposed language can be summarized as follows:

Write the mathematical decisions. State what may be reconstructed. Keep the theorem fixed. Accept only checked evidence. Preserve enough of that evidence to explain and replay the proof.

The ProveIt examples show why all five clauses matter. Absolute-value folding removes unnecessary sign management. Factorial decomposition benefits from either a short induction or a reusable parity view. Powerset induction needs a certified partition rather than informal reindexing. Differentiation needs the neighborhood argument to remain visible. Integer-square-root summation becomes clearer when a finite relation exposes exact division and nontruncating subtraction.

Leant provides a practical inspiration for filling local proof gaps without asking the author to spell out every term. Its existing design work also demonstrates that candidate identity and checking boundaries deserve careful engineering. Reason proposes to organize those capabilities around the units a mathematician actually wants to communicate: definitions, invariants, reductions, and justified transitions between representations.

The result should not be a language in which rigor is optional. It should be a language in which rigor is present throughout, but its repeated implementation details are displayed only when they help the author or reader.

A A compact grammar and an elaboration trace

A.1 Core grammar sketch

The grammar below describes the logical shell. The term nonterminal delegates to the host mathematical expression elaborator; method denotes a registered typed method. Domain commands in the worked examples are method-specific surface extensions, not an assertion that this core grammar alone parses every listing.

```
proof      ::= "proof" statement* "end"
statement  ::= "assume" binders
            | "let" name [":" term] ":" term
            | "claim" name ":" term justification
            | "suffices" term "via" term proof
            | "obtain" patterns "using" term proof
            | "cases" term branch+
            | "induct" name ["generalizing" names] branch+
            | "calc" calc_step+
            | "conclude" [term] justification
justification ::= "exact" term
                | ["from" names] ["using" methods]
                  "by" method ["with budget" budget]
                | proof
branch       ::= "case" pattern ["with" names] ":" proof_body
```

A real grammar must settle layout, precedence, binder syntax, and error recovery. The sketch specifies the relevant scope-producing constructs; it is not a parser implementation. Method names are resolved in a registry, while ordinary identifiers resolve in the Lean context. Both resolutions are recorded.

A.2 A complete structural lowering trace

For the application theorem of Section 3, freeze the target

$$G = (A \rightarrow B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C.$$

The assume line introduces $f : A \rightarrow B \rightarrow C$, $g : A \rightarrow B$, and $x : A$. The remaining obligation has target C . Structural synthesis proposes the following derivation:

$$\frac{g : A \rightarrow B \quad x : A}{g x : B}, \quad \frac{f : A \rightarrow B \rightarrow C \quad x : A}{f x : B \rightarrow C}, \quad \frac{f x : B \rightarrow C \quad g x : B}{f x (g x) : C}.$$

The worker checks the candidate in the exact context. The core then closes the three binders to obtain

$$\lambda f. \lambda g. \lambda x. f x (g x) : G.$$

The assembled theorem is checked again. The same target paired with the candidate $f x x$ fails unless the types actually make the final x an inhabitant of B . The system cannot repair that failure by changing B to A , because G is already frozen.

A.3 A typed bridge instead of a suggestive phrase

Suppose the current goal is G , and the author writes suffices B . Without a checked bridge, this is only a proposal. With $r : B \rightarrow G$, the transition has exact semantics. A domain method can synthesize r , but must return it for checking. This single pattern supports many mathematical reductions: proving equality after an injective map, reducing to a normal form, establishing a stronger estimate, or transporting a result along an equivalence.

The author need not write the full bridge term every time. The method registry can supply it from a named theorem and local data. The important property is that a bridge exists in the final evidence and preserves the original target.

B Failure cases and required responses

Attempt	Required response
A claim uses its own name while being proved.	Reject the scope violation; the name enters the context only after its proof is checked.
A witness escapes an existential-elimination block.	Reject or require an explicit existential conclusion or appropriate data-level construction.
A search result proves a nearby but different proposition.	Reject at the frozen target; present a statement change only as a separate author edit.
A stale worker returns a proof with identical printed text.	Reject the stale origin, or rebuild and recheck the term in the new exact environment.
A tactic closes the goal with an unapproved axiom.	Fail publication policy even if the term is typable relative to that axiom.
A natural subtraction is cast without a bound.	Generate and prove the order obligation, or stop the transport.
A finite sum is reindexed through a noninjective map.	Require a bijection, an appropriate fiber theorem, or multiplicity accounting.
A derivative step uses only point-wise equality at the point.	Request neighborhood equality or another valid derivative-congruence theorem.
Search finds no term within its budget.	Return a bounded miss, not a proof of negation.

Attempt	Required response
A heuristic reports that a synthesized program looks correct.	Use the result only for the role justified by its specification; do not promote it to a theorem.
A proof is correct but ignores the named mathematical method.	Accept only under a permissive evidence annotation, or change the explanation after explicit review.
A cached certificate hash matches.	Use it to locate an artifact; perform the checks required by the replay policy.

C Source map and reproducibility notes

All repository references below were accessed on 5 September 2026. Paths are relative to the repository root. The bibliography gives direct links. These notes identify the inspected material without implying that its entire repository was audited.

C.1 Provelt

The directory prefix for the first three files is

```
| Analysis/FabiusFunction/Lean/FabiusFunction/
```

`Basic.lean` supplied `BoundedFabius`, `fabiusReal`, `rvachevUp`, `rvachevUp_even`, and examples of real/complex transport.

`Arithmetic.lean` supplied `choose_succ_two`, `oddDoubleFactorial`, and `factorial_two_mul_eq`.

`AffineDifferenceOrbit.lean` supplied the affine-difference and affine-cube definitions and the following main theorem names:

```
| affineDifference_iterate_apply
| iteratedDeriv_eq_affineDifference_iterate_on
```

The integer-square-root example is in

```
| NumberTheory/IntegerSums/Lean/IntegerSums/FloorSqrtSum.lean
```

The relevant declarations include `floorSqrtSumClosedForm`, `floorSqrtSumClosedForm_cast`, and `sum_floor_sqrt_eq`. The public theorem is in natural numbers, while a private rational formula supports the implementation.

The reviewed `lean-toolchain` specifies `leanprover/lean4:v4.32.0`. The source license is MIT No Attribution; the small excerpt in this article is nevertheless attributed [6]. No commit SHA or build result is claimed.

C.2 Leant

The reviewed sources are `README.md`, `docs/synth-internals.md`, `docs/candidate-quality.md`, `src/Leant/Synth/Verification.hs`, `src/Leant/Synth/Fragment.hs`, and

```
| docs/Leant_Djex_Lean4_Rewrite_Analysis/
| Leant_Djex_Lean4_Rewrite_Analysis.tex
```

The last is a design study, not evidence that its proposed native rewrite has been implemented. In the Haskell code, `verifyCandidateGroups` and `candidateVerificationProgram` are particularly relevant interfaces. No runtime transcript or acceptance statistic from the repository is presented here as an independently reproduced result.

C.3 What can be reproduced from this archive

The article can be rebuilt with XeLaTeX. Its bibliography is embedded in the main source. The Python companion uses the standard library and can be tested with

```
| python -m unittest discover -s supplement -p 'test_*.py' -v
```

The archive includes the test transcript and build instructions. It does not include a Lean project or a completed Reason implementation. A future benchmark release should additionally include pinned Git commits, the full dependency lock, all new helper lemmas, theorem-by-theorem transcripts, and search-disabled replay results.

References

- [1] ProveIt contributors. *Basic.lean*, Fabius-function development. Repository source, main branch, accessed 5 September 2026. <https://github.com/VladimirReshetnikov/ProveIt/blob/main/Analysis/FabiusFunction/Lean/FabiusFunction/Basic.lean>.
- [2] ProveIt contributors. *Arithmetic.lean*, Fabius-function development. Repository source, main branch, accessed 5 September 2026. <https://github.com/VladimirReshetnikov/ProveIt/blob/main/Analysis/FabiusFunction/Lean/FabiusFunction/Arithmetic.lean>.
- [3] ProveIt contributors. *AffineDifferenceOrbit.lean*. Repository source, main branch, accessed 5 September 2026. <https://github.com/VladimirReshetnikov/ProveIt/blob/main/Analysis/FabiusFunction/Lean/FabiusFunction/AffineDifferenceOrbit.lean>.
- [4] ProveIt contributors. *FloorSqrtSum.lean*. Repository source, main branch, accessed 5 September 2026. <https://github.com/VladimirReshetnikov/ProveIt/blob/main/NumberTheory/IntegerSums/Lean/IntegerSums/FloorSqrtSum.lean>.
- [5] ProveIt contributors. *lean-toolchain*. Accessed 5 September 2026. <https://github.com/VladimirReshetnikov/ProveIt/blob/main/lean-toolchain>.
- [6] ProveIt contributors. *MIT No Attribution License*. Accessed 5 September 2026. <https://github.com/VladimirReshetnikov/ProveIt/blob/main/LICENSE>.
- [7] Leant contributors. *Leant README*. Main branch, accessed 5 September 2026. <https://github.com/VladimirReshetnikov/Leant/blob/main/README.md>.
- [8] Leant contributors. *Synthesis internals*. Accessed 5 September 2026. <https://github.com/VladimirReshetnikov/Leant/blob/main/docs/synth-internals.md>.
- [9] Leant contributors. *Candidate quality*. Accessed 5 September 2026. <https://github.com/VladimirReshetnikov/Leant/blob/main/docs/candidate-quality.md>.
- [10] Leant contributors. *Leant.Synth.Verification*. Haskell source, accessed 5 September 2026. <https://github.com/VladimirReshetnikov/Leant/blob/main/src/Leant/Synth/Verification.hs>.
- [11] Leant contributors. *Leant.Synth.Fragment*. Haskell source, accessed 5 September 2026. <https://github.com/VladimirReshetnikov/Leant/blob/main/src/Leant/Synth/Fragment.hs>.
- [12] Leant contributors. *Leant/Djex Lean 4 Rewrite Analysis*. Design study, accessed 5 September 2026. https://github.com/VladimirReshetnikov/Leant/blob/main/docs/Leant_Djex_Lean4_Rewrite_Analysis/Leant_Djex_Lean4_Rewrite_Analysis.tex.
- [13] Lean developers. *Lean Language Reference: Tactic Proofs*. Online reference, accessed 5 September 2026. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/>.
- [14] Lean developers. *Lean Language Reference: Validating Proofs*. Online reference, accessed 5 September 2026. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.

- [15] Lean developers. *Lean 4.33.0 release notes*. Accessed 5 September 2026. <https://lean-lang.org/doc/reference/latest/releases/v4.33.0/>.
- [16] Makarius Wenzel and Isabelle contributors. *The Isabelle/Isar Reference Manual*. Online edition, accessed 5 September 2026. <https://isabelle.in.tum.de/doc/isar-ref.pdf>.
- [17] Adam Grabowski, Artur Korniłowicz, and Adam Naumowicz. *Mizar in a Nutshell*. Journal of Formalized Reasoning, 3(2), 2010. <https://jfr.unibo.it/article/download/1980/1356>.
- [18] Adrian De Lon, Peter Koepke, Anton Lorenzen, Adrian Marti, Marcel Schütz, and Makarius Wenzel. *The Isabelle/Naproche Natural Language Proof Assistant*. CADE 2021, LNCS 12699, pp. 614–624. DOI: 10.1007/978-3-030-79876-5_36. Author-hosted version: https://adelon.net/share/2021_IsabelleNaproche.pdf.
- [19] Patrick Massot and contributors. *Verbose Lean 4*. Repository and project description, accessed 5 September 2026. <https://github.com/PatrickMassot/verbose-lean4>.
- [20] Jannis Limperg and contributors. *Aesop: White-box automation for Lean 4*. Repository and documentation, accessed 5 September 2026. <https://github.com/leanprover-community/aesop>.